

CHAPTER I

INTRODUCTION

1.1 PREAMBLE

The Software Development Life Cycle (SDLC) is a structured framework that governs the systematic development, deployment, and maintenance of software systems. It provides a disciplined approach to software engineering by dividing the development process into well-defined phases such as requirement analysis, system design, implementation, testing, deployment, and maintenance. The primary goal of SDLC is to ensure that software systems are delivered with high quality, within budget, and within the expected timeline.

Historically, software development was manageable through manual processes because systems were relatively small, standalone, and limited in scope. However, in the modern digital era, software systems have become highly complex, distributed, data-driven, and tightly integrated with cloud platforms and enterprise infrastructures. Applications are expected to scale dynamically, support real-time users, ensure security, and evolve continuously in response to changing business needs.

Despite the evolution of development methodologies such as Agile and DevOps, many SDLC activities remain heavily dependent on human effort. Requirement analysis, design documentation, code review, and testing are largely manual and knowledge-intensive. Literature consistently reports that unclear requirements and late defect detection are among the leading causes of software project failure [1] Ferrari et al., 2024; [2] Raghi et al., 2024. These limitations motivate the need for intelligent, automated, and context-aware support systems for SDLC optimization.

1.1.1 Evolution of Software Development Practices

Software development practices have evolved significantly over the last few decades. Early development followed linear models such as the Waterfall model, where each phase was completed sequentially. While the Waterfall model provided structured documentation and control, it lacked flexibility and failed to accommodate changing requirements, often resulting in costly rework.

To overcome these limitations, iterative and incremental models such as the Spiral model were introduced. Agile methodologies further transformed software development by emphasizing adaptability, customer collaboration, and continuous feedback. DevOps extended Agile principles by integrating development and operations, enabling continuous integration and continuous deployment pipelines.

Although these methodologies improved delivery speed and collaboration, they did not eliminate the heavy reliance on human expertise. Requirement interpretation, design decision-making, and testing strategies still depend on manual analysis. Studies indicate that even in Agile and DevOps environments, requirement ambiguity and documentation inconsistencies persist [3] Arshad et al., 2025.

Table 1.1: EVOLUTION OF SDLC MODELS AND THEIR LIMITATIONS

SDLC Model	Key Characteristics	Major Limitations
Waterfall	Sequential phases	No flexibility
Spiral	Risk-driven	Complex management
Agile	Iterative development	Human-dependent analysis
DevOps	CI/CD integration	Limited intelligence

1.1.1.a Role of Artificial Intelligence in SDLC

Artificial Intelligence (AI) has emerged as a powerful tool to address the growing complexity of software engineering. Early AI applications in SDLC focused on rule-based expert systems and traditional machine learning techniques such as Naïve Bayes and Decision Trees for requirement classification and defect prediction [4] K & Samuel, 2022. These approaches provided partial automation but were limited by domain dependency and lack of semantic understanding.

With advancements in Natural Language Processing (NLP), AI systems became capable of analyzing requirement documents written in natural language. Transformer-based models significantly improved contextual understanding and enabled tasks such as requirement extraction, UML generation, and code suggestion. However, most AI solutions were designed to support individual SDLC phases, leading to fragmented workflows.

Recent studies highlight the potential of Large Language Models (LLMs) to perform complex reasoning tasks across software engineering domains [1] Ferrari et al., 2024. Nevertheless, direct use of LLMs without project-specific grounding results in hallucinated outputs and reduced reliability, limiting their adoption in critical SDLC tasks.

1.2 DETAILED PROBLEM DEFINITION

Despite the availability of modern development methodologies and tools, the SDLC continues to face persistent challenges. Software Requirement Specification (SRS) documents are written in natural language and are inherently ambiguous, incomplete, and inconsistent. Manual analysis of such documents is time-consuming and prone to misinterpretation, which directly impacts system design and implementation.

Design activities such as UML diagram creation and architectural modeling require significant manual effort and often lack alignment with evolving requirements. During implementation, developers rely heavily on personal expertise and informal documentation, leading to inconsistencies and reduced traceability. Testing is frequently delayed until later stages, resulting in late defect discovery and increased project cost.

Although AI-based tools exist for requirement analysis, design generation, and testing, these tools operate independently and do not share a unified understanding of project context. Literature surveys emphasize the absence of integrated AI frameworks capable of reasoning across multiple SDLC artifacts [2] Raghi et al., 2024; [3] Arshad et al., 2025.

1.3 PURPOSE OF THIS WORK

The purpose of this work is to design and develop a cloud-enabled AI framework that provides intelligent, context-aware assistance throughout the Software Development Life Cycle. The proposed system aims to overcome the limitations of isolated AI tools by integrating requirement analysis, design generation, implementation support, testing assistance, and maintenance insights within a unified framework.

The project specifically focuses on using Retrieval-Augmented Generation (RAG) to ground AI outputs in project-specific data, thereby improving accuracy, traceability, and reliability. The system is designed as an AI-assisted decision-support framework rather than a fully autonomous development system.

1.4 SIGNIFICANCE OF THE STUDY

The significance of this study lies in its strong practical as well as academic contributions. From an industry perspective, the proposed framework provides a systematic approach to automating and enhancing requirement analysis by minimizing manual intervention. By leveraging Retrieval-Augmented Generation (RAG), the framework improves the clarity, consistency, and completeness of software requirements, thereby reducing ambiguity that often leads to project delays or failures. Additionally, the framework enables early identification of potential risks, inconsistencies, and missing requirements during the initial phases of the software development life cycle (SDLC). This early risk detection helps organizations reduce rework costs, improve development efficiency, and enhance overall project quality and reliability.

From an academic perspective, this study contributes to the growing body of research on the application of large language models (LLMs) and RAG-based architectures in software engineering. It empirically demonstrates how RAG can be effectively integrated into requirement engineering tasks such as requirement validation, classification, and traceability. By addressing the research gap highlighted in recent literature [3] Arshad et al., 2025, this work extends existing theoretical knowledge and provides a practical reference architecture for future studies. Furthermore, the findings encourage interdisciplinary research by bridging artificial intelligence and software engineering, paving the way for future advancements in intelligent requirement engineering systems.

1.5 BACKGROUND STUDY

Early research in SDLC automation relied on classical machine learning techniques that required extensive labelled datasets and domain-specific tuning. NLP-based approaches improved textual analysis but lacked integration across SDLC phases. Recent research highlights the effectiveness of LLMs in software engineering;

however, these models struggle with hallucination when used without contextual grounding [1] Ferrari et al., 2024.

Retrieval-Augmented Generation has emerged as a promising approach to overcome these limitations by combining semantic retrieval with generative reasoning. RAG enables AI systems to reason over actual project data rather than relying solely on pre-trained knowledge.

1.6 NON-TECHNICAL EVALUATION OF EXISTING SOLUTIONS

Existing SDLC support tools include requirement analysis systems, UML generators, code assistants, and testing frameworks. While these tools improve productivity, they operate as standalone solutions and lack shared context. They require extensive human validation and do not provide holistic SDLC optimization.

Table 1.2: TRADITIONAL SDLC VS AI-ASSISTED SDLC

Aspect	Traditional SDLC	AI-Assisted SDLC
Requirement Analysis	Manual	AI-supported
Design	Manual UML	Automated UML
Testing	Late & manual	Early AI-assisted

1.7 CHALLENGES TO BE RESOLVED AND GAP IDENTIFIED

The literature identifies several unresolved challenges:

- Lack of unified AI frameworks across SDLC phases
- Poor traceability between requirements, design, and code
- Hallucination issues in direct LLM usage
- Absence of continuous feedback mechanisms

Table 1.3: RESEARCH GAPS AND PROPOSED SOLUTIONS

Identified Gap	Impact	Proposed Approach
Phase isolation	Fragmentation	Unified framework

No grounding	Hallucination	RAG-based retrieval
No feedback	Static systems	Continuous learning

1.8 AIM AND OBJECTIVES OF THE PROPOSED WORK

Aim

To design and implement a cloud-enabled, RAG-based AI framework for optimizing the Software Development Life Cycle.

Objectives

- To analyze SRS documents using AI techniques
- To extract contextual knowledge from source code
- To design a Retrieval-Augmented Generation pipeline
- To generate automated design artifacts
- To support testing and maintenance activities

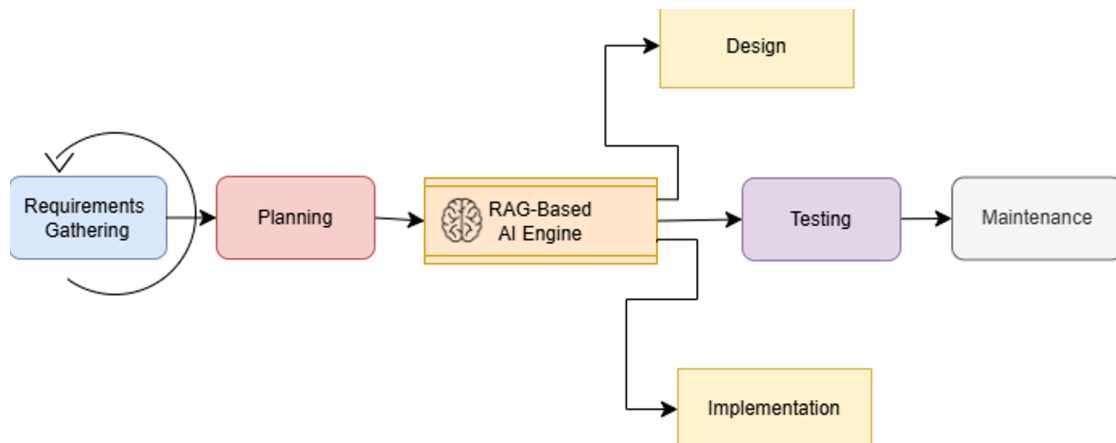


Figure 1.1: AI Enabled SDLC Optimization Framework

1.9 APPLICATIONS OF THE PROJECT

The proposed system can be applied in enterprise software development, Agile and DevOps environments, academic research, requirement engineering automation, and large-scale cloud-based projects.

1.10 INITIAL ASSUMPTIONS

The system assumes that SRS documents are reasonably structured, source code follows standard practices, and cloud infrastructure is available.

1.11 DEFINITIONS OF TECHNICAL TERMS

- SDLC: Software Development Life Cycle
- AI: Artificial Intelligence
- LLM: Large Language Model
- RAG: Retrieval-Augmented Generation

1.12 ORGANIZATION OF THE REMAINING CHAPTERS

Chapter II presents the literature review. Chapter III discusses system requirements. Chapter IV explains system design. Chapter V describes methodology and implementation. Chapter VI presents results and discussion, followed by conclusions and future scope.

1.13 SUMMARY OF CHAPTER I

This chapter provided a detailed introduction to the proposed research work by establishing its conceptual, technical, and practical foundations. It began with an overview of the Software Development Life Cycle (SDLC), highlighting its importance in delivering high-quality software systems through structured and well-defined phases. The discussion emphasized how increasing software complexity, scalability requirements, and continuous evolution have exposed the limitations of traditional, human-intensive SDLC practices.

The chapter traced the evolution of software development methodologies from linear models such as Waterfall to modern Agile and DevOps approaches. While these methodologies improved flexibility and delivery speed, the chapter identified that critical activities such as requirement analysis, design validation, and testing still rely heavily on manual effort and expert judgment. Persistent issues such as requirement ambiguity, delayed defect detection, and poor traceability were discussed as major contributors to software project failures.

The role of Artificial Intelligence (AI) in addressing these challenges was then introduced, outlining the progression from rule-based systems to machine learning,

NLP, and large language models (LLMs). Although recent advances in LLMs have shown promise in automating software engineering tasks, the chapter highlighted key limitations such as hallucination, lack of project-specific grounding, and fragmented AI tool usage across SDLC phases.

A detailed problem definition was presented, clearly identifying the absence of an integrated, context-aware AI framework capable of supporting multiple SDLC phases in a unified manner. The purpose and significance of the proposed work were articulated, emphasizing both its industrial relevance in improving productivity and reducing risk, and its academic contribution in addressing research gaps through the application of Retrieval-Augmented Generation (RAG).

The background study and evaluation of existing solutions further reinforced the need for a RAG-based approach that combines semantic retrieval with generative reasoning to ensure accuracy, traceability, and reliability. The chapter also identified key research gaps, challenges to be resolved, and the aim and objectives of the proposed framework.

Finally, the chapter outlined the applications, assumptions, technical definitions, and organization of the remaining chapters, thereby establishing a clear roadmap for the rest of the thesis. Overall, Chapter I lays a strong foundation for the proposed research by clearly defining the motivation, scope, and direction of a unified, AI-enabled SDLC optimization framework.

CHAPTER II

LITERATURE REVIEW

2.1 PREAMBLE

This chapter presents a detailed literature review of recent research focused on the application of Artificial Intelligence (AI) and cloud computing technologies for optimizing the Software Development Life Cycle (SDLC). With the rapid growth of complex software systems and the increasing adoption of cloud-native architectures, traditional SDLC practices have become insufficient to meet modern demands for scalability, reliability, and speed.

Recent studies emphasize that inefficiencies in early SDLC phases—particularly requirement analysis and system design—often propagate errors into later stages, leading to increased cost and reduced software quality. As a result, researchers have explored AI-based solutions to automate and enhance SDLC activities. This chapter surveys recent literature (2023 onwards) to analyze current methodologies, compare results, identify limitations, and establish research gaps that motivate the proposed cloud-enabled, Retrieval-Augmented Generation (RAG)-based AI framework.

2.2 SCOPE AND SELECTION CRITERIA

The scope of this literature review includes AI and cloud-based approaches applied across all major SDLC phases:

- Requirement Engineering
- Design Automation
- Code Analysis and Review
- Test Case Generation
- Deployment and Maintenance

The primary corpus is derived from your systematic review process, where 22 high-quality research papers were selected from an initial pool of over 300 articles using

PRISMA-based filtering. Only peer-reviewed journal and conference papers published from 2023 onwards are considered in this chapter to ensure alignment with current research trends, especially in Large Language Models (LLMs) and Generative AI.

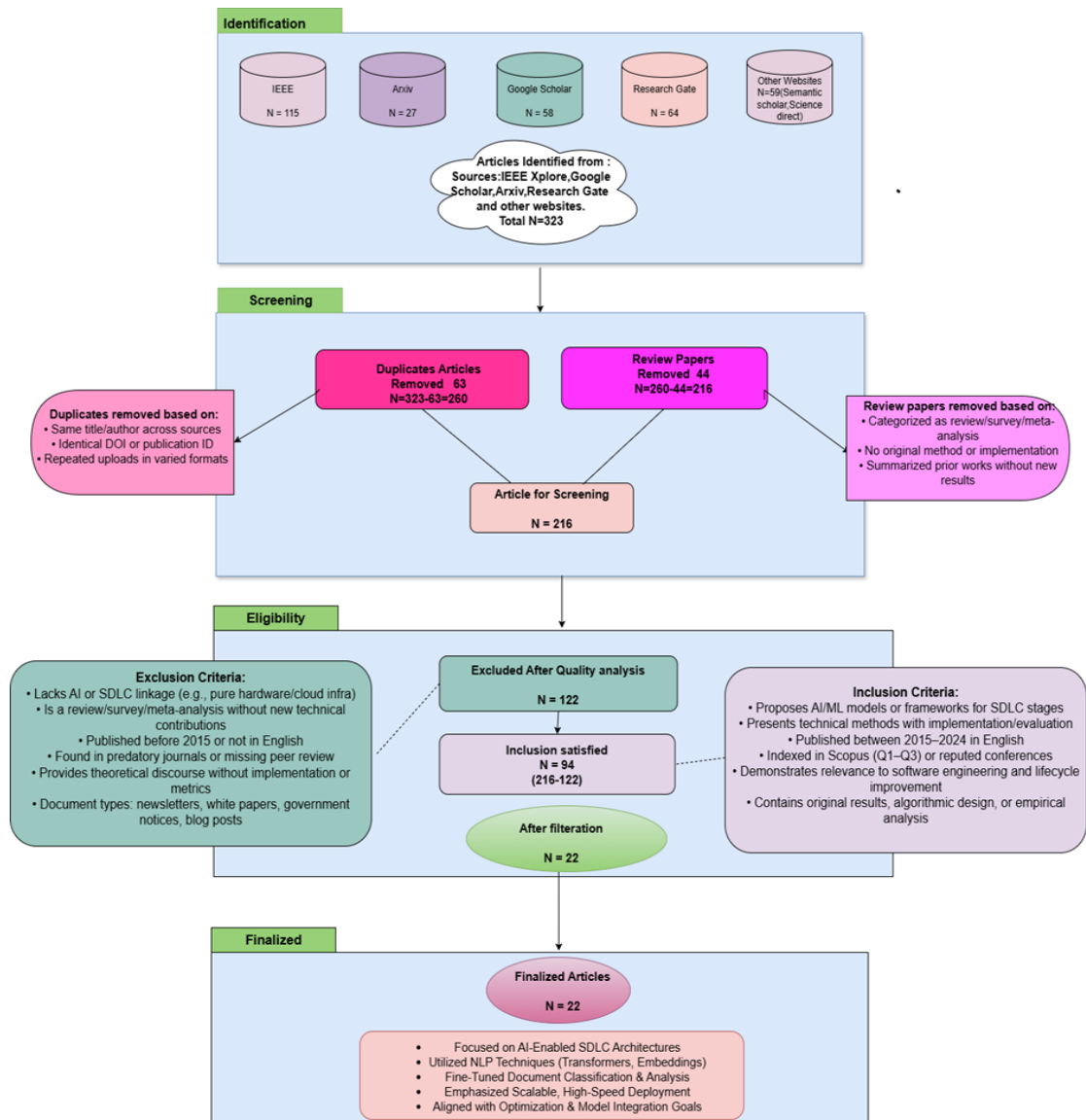


Figure 2.1: Flowchart Representing the PRISMA standard applied for paper selection

The inclusion criteria are:

1. Direct relevance to SDLC optimization
2. Clear methodology and experimental evaluation
3. Use of AI, ML, LLM, cloud, or hybrid techniques

Survey-only papers, opinion articles, and pre-2023 studies were excluded.

2.3 THEMATIC ORGANIZATION OF REVIEWED WORK

To improve readability and analytical clarity, the reviewed literature is organized into thematic categories aligned with SDLC phases and cross-cutting concerns:

- 2.3.1 Requirement Engineering and Natural Language Processing
- 2.3.2 Design Automation and UML Generation
- 2.3.3 Code Analysis, Generation, and Review
- 2.3.4 Test Case Generation and Validation
- 2.3.5 Cloud Integration and Scalable Inference
- 2.3.6 Retrieval-Augmented Generation and Hallucination Mitigation
- 2.3.7 Ethics, Explainability, and Human-in-the-Loop

Each subsection discusses multiple recent papers, summarizes key contributions, and critically analyzes their limitations.

2.3.1 Requirement Engineering and Natural Language Processing

Requirement engineering is widely regarded as the most critical and error-prone phase of SDLC. Recent research consistently reports that ambiguous or incomplete requirements are a leading cause of software project failure.

2.3.1.a AI-Based Requirement Extraction

Recent studies (2023–2024) have applied transformer-based NLP models to automate requirement extraction from Software Requirement Specification (SRS) documents. These models demonstrate improved semantic understanding compared to classical machine learning techniques, enabling better identification of functional and non-functional requirements.

[1]Ferrari et al. (2024) investigated LLM-based requirement interpretation and reported significant improvements in contextual understanding and ambiguity detection. However, the study highlighted hallucination issues when LLMs were used without grounding in project-specific data.

2.3.1.b Comparison with Classical ML

While classical ML models such as Naïve Bayes and SVM remain effective for structured datasets, recent literature emphasizes their limitations in handling unstructured and domain-specific language. Hybrid approaches combining lightweight classifiers for structure and LLMs for semantics have been proposed as more robust solutions.

2.3.1.c Critical Analysis

Although recent AI-based systems improve requirement clarity, most solutions:

- Operate independently of design and testing phases
- Lack traceability mechanisms
- Require significant human validation

These limitations motivate integrated, lifecycle-wide AI frameworks.

2.3.2 Design Automation and UML Generation

Design automation aims to reduce manual effort in creating UML diagrams and architectural documentation.

2.3.2.a Transformer and LLM-Based UML Generation

Recent studies (2023–2025) demonstrate that LLMs can generate UML diagrams from textual requirements, significantly reducing documentation effort. [5]Alksasbeh et al. (2023) proposed a transformer-based design pipeline that converts requirements into intermediate semantic representations before UML generation.

[1]Ferrari et al. (2024) showed that LLM-generated UML artifacts improve productivity but often lack architectural constraints unless supported by contextual examples.

2.3.2.b Limitations Identified

Despite promising results, existing design automation systems:

- Do not integrate with code-level artifacts
- Fail to enforce non-functional requirements

- Depend heavily on prompt quality

Recent research suggests that retrieval of prior designs and code examples improves UML accuracy, paving the way for RAG-based design automation.

2.3.3 Code Analysis, Generation, and Review

Recent advances in AI have significantly influenced code analysis, generation, and review processes within the SDLC. Traditional static analysis tools rely on predefined rules and metrics, which limits their adaptability to evolving codebases. As a result, researchers have explored machine learning and deep learning techniques to improve code quality assessment and defect prediction.

2.3.3.a AI-Based Defect Prediction

Several studies published between 2023 and 2025 focus on defect prediction using ensemble learning techniques.[3]Arshad et al. (2025) evaluated Random Forest, Gradient Boosting, and hybrid ensemble models using public datasets such as PROMISE and NASA MDP. Their results demonstrate a 10–15% improvement in ROC-AUC compared to individual classifiers, indicating that ensemble approaches are more robust for identifying fault-prone modules.

These studies emphasize that static code metrics—such as cyclomatic complexity, coupling, and code churn—remain strong predictors of defects when combined with AI models. However, they also note that defect prediction models are highly dataset-dependent and often lack explainability, which limits developer trust.

2.3.3.b LLM-Based Code Generation and Review

Recent literature also explores the use of Large Language Models for code generation, refactoring, and review. [2]Raghi et al. (2024) examined the effectiveness of LLM-assisted code review in enterprise environments and reported measurable improvements in developer productivity and review turnaround time. Nevertheless, the study cautions that LLM-generated suggestions may introduce logical or security flaws if not carefully validated.

Other 2023–2024 studies indicate that LLM-based code generation performs best when provided with in-project context, such as existing code snippets and architectural constraints. This reinforces the importance of storing code embeddings in

a centralized knowledge base and retrieving relevant context during code analysis and review.

2.3.3.c Critical Interpretation

Overall, the literature suggests that AI-based code analysis tools are effective as decision-support systems rather than fully autonomous solutions. Integrating classical ML for numeric analysis with LLM-based reasoning for contextual understanding emerges as a promising hybrid approach.

2.3.4 Test Case Generation and Validation

Software testing is a resource-intensive phase of SDLC, and delays in testing often lead to late defect detection and increased project cost. Recent research has focused on AI-driven approaches to automate test-case generation and improve test coverage.

2.3.4.a AI-Based Test Case Generation

Studies published in 2023 and 2024 report that AI-generated test cases derived from requirements or source code significantly improve coverage metrics. Machine learning models and LLMs have been used to generate test scenarios, boundary cases, and regression tests. However, many studies also report that automatically generated test cases may lack domain relevance or produce redundant scenarios.

Hybrid approaches combining static analysis, defect prediction, and AI-based test generation have shown better results. For example, using defect prediction outputs to prioritize high-risk modules before generating test cases improves testing efficiency.

2.3.4.b Limitations and Human-in-the-Loop Requirement

Despite automation benefits, recent literature strongly emphasizes the necessity of human-in-the-loop validation. Developers are required to review, refine, and approve AI-generated test cases to ensure correctness and relevance. This highlights that AI-driven testing should complement, not replace, human expertise.

2.3.5 Cloud Integration and Scalable Inference

Cloud computing plays a crucial role in enabling AI-driven SDLC systems, particularly when dealing with large-scale projects and complex AI models.

2.3.5.a Role of Cloud Infrastructure

Recent cloud-enabled AI SDLC frameworks (2024–2025) highlight several advantages of cloud integration:

- Elastic compute resources for training and inference
- Centralized storage of SDLC artifacts
- Real-time collaboration among distributed teams

Cloud platforms also support scalable embedding generation and vector search, which are essential for Retrieval-Augmented Generation pipelines.

2.3.5.b Challenges in Cloud-Based AI SDLC Systems

Despite these benefits, studies also identify practical challenges, including:

- High operational cost of large-scale AI inference
- Latency issues in real-time interactions
- Data privacy and intellectual property concerns

Recent research suggests hybrid cloud or on-premise deployments combined with privacy-preserving techniques as potential solutions.

2.3.6 Retrieval-Augmented Generation (RAG) and Hallucination

Mitigation

Hallucination is a major concern when applying LLMs to technical domains such as software engineering. Recent studies identify Retrieval-Augmented Generation as one of the most effective strategies for mitigating hallucination.

2.3.6.a RAG in Software Engineering

RAG pipelines retrieve relevant document or code chunks from a vector database and supply them as context to the LLM during generation. Studies published between 2023 and 2025 consistently report that RAG-based systems outperform direct LLM prompting in terms of factual accuracy, relevance, and traceability.

Research also shows that the choice of chunking strategy significantly affects retrieval performance. Paragraph-level chunking works well for requirement documents, while function-level or AST-based chunking is more effective for source code.

2.3.6.b Implications for SDLC Optimization

RAG enables AI systems to reason over project-specific knowledge, making it particularly suitable for SDLC tasks that require accuracy and accountability, such as requirement traceability and design validation.

2.3.7 Ethics, Explainability, and Human-in-the-Loop Considerations

Recent adoption-focused studies emphasize that ethical considerations, explainability, and accountability are critical for deploying AI in software engineering.

2.3.7.a Explainability and Trust

[20]Russo (2023) highlights that lack of transparency in AI-generated outputs reduces developer trust and hinders enterprise adoption. Systems that provide provenance information—such as which documents or code snippets influenced a generated output—are more likely to be accepted.

2.3.7.b Human-in-the-Loop Systems

Most recent studies agree that AI-assisted SDLC tools must operate in a human-in-the-loop paradigm. Developers remain responsible for final decisions, while AI systems provide recommendations, insights, and automation support.

2.4 COMPARATIVE ANALYSIS OF RECENT AI-BASED SDLC APPROACHES

This section presents a comparative evaluation of recent AI-based SDLC optimization approaches. The comparison is structured across hypotheses, parameters, methodologies, and reported results to highlight strengths, limitations, and research trends.

TABLE 2.1: ANALYTICAL COMPARISON OF RECENT SDLC STUDIES

Author(s) & Year	Technology Used	SDLC Phase / Coverage	Key Contributions	Limitations
[3]Naveed Arshad (2025)	AI-Driven SDLC Framework	End-to-End SDLC	Integrated AI and performance optimization across all SDLC phases; improved scalability and fault tolerance	High computational requirements
[6]Talha A. Butt (2025)	Cloud-Enabled AI Validation	Deployment, Monitoring	Validated AI-SDLC using GitHub Copilot and Prometheus; improved scalability and system efficiency	Dependency on cloud infrastructure
[7]Muhammad Iqbal (2025)	AI-Based Performance Benchmarking	Testing, Maintenance	Achieved over 90% sensitivity in SDLC performance benchmarks	Requires skilled AI professionals and costly infrastructure
[8]Nabeel Faruqui et al. (2024)	Transformer-Based Decision Support (AI-Analyst)	Planning, Cost Optimization	Reduced business cost and improved SDLC decision-making using	High computational cost and limited dataset generalization

			transformer models	
[9]Asif Qumer Gill (2025)	Decision-Centric Agile SDLC	Requirements, Design	Integrated decision science to manage bias, governance, and trust in AI systems	Lacks empirical validation and quantitative metrics
[10]Bharath Madupati (2024)	Generative AI (GPT-4, Codex)	Implementation , Testing	Improved automation in code generation, debugging, and CI/CD pipelines	Ethical bias concerns and lack of quantitative evaluation
[1]Alessandro Ferrari et al. (2024)	Large Language Models (LLMs)	Requirements	Enhanced requirement interpretation and ambiguity detection	Hallucination when project context is absent
[4]Vineetha K. V. & Samuel (2022)	Naïve Bayes + NLP	Requirements	Automated actor and use-case extraction from SRS documents	Limited semantic depth and domain dependency
[12]Yuan Zhang & Li (2024)	LLM-Based Requirement Classification	Requirements	Demonstrated superior semantic understanding over traditional NLP	Explainability and transparency challenges

[5]Mohammad Alksasbeh et al. (2023)	Transformer-Based UML Generation	Design	Generated UML diagrams using intermediate semantic representations	Weak handling of non-functional requirements
[13]Rui Dias et al. (2023)	NLP + Naïve Bayes + PlantUML	Design	Automated UML use-case diagram generation from user stories	Poor support for multilingual input and complex relationships
[14]Humberto Cervantes et al. (2024)	LLM + Attribute-Driven Design (ADD)	Architecture	Improved architectural documentation clarity and traceability	Context loss across iterative design cycles
[2]Karthik Raghi et al. (2024)	LLM-Assisted Code Review	Implementation	Improved developer productivity and review turnaround time	Risk of logical and security flaws
[15]Jian Wang et al. (2023)	LLM-Based Software Testing	Testing	Enhanced test coverage and automated test generation	Requires strong human-in-the-loop validation
[16]Venkatesh Saravanan et al. (2025)	Generative AI for Debugging	Maintenance	Improved debugging efficiency and	Over-reliance on AI automation

			code generation	
[17]Chaima Boufaied et al. (2025)	Security Debt Analysis	Security	Identified accumulation of unresolved vulnerabilities due to delivery pressure	Qualitative study with limited automation
[20]Danilo Russo (2023)	Explainable & Ethical AI	Cross-Cutting	Highlighted trust, explainability, and adoption challenges of generative AI	Organizational and cultural resistance

This table presents a comprehensive review of recent research works (2022–2025) that apply Artificial Intelligence techniques across various phases of the Software Development Life Cycle. It highlights the technologies used, such as Large Language Models, transformers, generative AI, and traditional NLP methods, along with their targeted SDLC phases. The key contributions demonstrate improvements in automation, decision-making, performance optimization, and requirement analysis. However, the table also identifies common limitations, including high computational costs, ethical concerns, lack of explainability, and dependence on skilled professionals. Overall, the comparison reveals both the potential and challenges of integrating AI into modern software engineering practices.

TABLE 2.2: PARAMETERS COMPARISON – DATASETS, MODELS, AND METRICS

Study	Dataset Type	AI Technique	Metrics Used	Cloud Support
[1]Ferrari et al., 2024	SRS & design docs	LLM	Relevance	Partial

[3]Arshad et al., 2025	PROMISE, NASA MDP	RF, GBM	ROC-AUC	No
[2]Raghi et al., 2024	Enterprise code repos	LLM	Productivity	No
RAG-based SDLC, 2024	SRS + code chunks	RAG + LLM	Relevance, Traceability	Yes

The comparison highlights a clear division: classical ML uses structured datasets and numeric metrics, while LLM and RAG-based systems rely on semantic relevance and contextual evaluation. Only cloud-enabled RAG systems address scalability.

TABLE 2.3: METHOD COMPARISON ACROSS SDLC PHASES

Method Type	Input	Processing	Output	Limitation
Classical ML	Structured SRS	Feature-based learning	Labels	Low semantic depth
NLP-based	Text	Parsing + embeddings	Entities	Limited reasoning
LLM-only	Raw text/code	Prompt-based generation	Text/code	Hallucination
RAG-based AI	Chunked artifacts	Retrieval + LLM	Grounded output	Setup complexity
Cloud AI Framework	Centralized artifacts	Distributed AI pipeline	Multi-phase output	Cost

This table illustrates the evolution of SDLC automation methods. RAG-based AI systems overcome major limitations of LLM-only approaches by grounding outputs in retrieved project knowledge.

2.4.1 THEMATIC REVIEW OF EXISTING SYSTEMS

2.4.1.a AI-Driven SDLC Optimization Frameworks

Recent research has increasingly focused on the development of AI-driven frameworks that embed intelligent automation across multiple phases of the Software

Development Life Cycle. These frameworks aim to overcome the inherent limitations of traditional SDLC models, such as rigid phase separation, high human dependency, delayed feedback, and limited scalability. By incorporating artificial intelligence techniques, researchers seek to enable adaptive decision-making, continuous optimization, and data-driven insights throughout the lifecycle.

[3]Arshad (2025) proposed a comprehensive AI-driven SDLC optimization framework that integrates artificial intelligence and performance optimization mechanisms across all major SDLC phases, including requirements analysis, system design, implementation, testing, deployment, and maintenance. The framework emphasizes end-to-end automation and intelligent coordination between development activities. Experimental evaluation in cloud environments demonstrated improved scalability, system robustness, and fault tolerance. However, the study also highlighted that the framework demands substantial computational resources and skilled AI expertise, which may limit adoption in small and medium-sized organizations.

Building upon this foundation, [6]Butt (2025) focused on validating AI-driven SDLC optimization in real-world cloud environments. By employing tools such as GitHub Copilot for code assistance and Prometheus for performance monitoring, the study demonstrated measurable improvements in system efficiency, scalability, and operational visibility within a Smart Logistics Management System. The findings underscore the importance of cloud-native infrastructure in realizing the full potential of AI-driven SDLC frameworks, while also raising concerns regarding cloud dependency and operational cost.

[7]Iqbal (2025) concentrated on evaluating the performance impact of AI integration within SDLC workflows. Through extensive benchmarking, the study reported over 90% sensitivity in performance metrics, indicating strong predictive and optimization capabilities of AI-enabled SDLC processes. Nevertheless, the research acknowledged challenges related to infrastructure cost, energy consumption, and the availability of skilled professionals, emphasizing the need for cost-aware and resource-efficient AI deployment strategies.

[8]Faruqui et al. (2024) introduced the AI-Analyst framework, a transformer-based SDLC decision-support system designed to optimize business cost and resource

allocation. Trained on a large SDLC-specific corpus, the framework demonstrated significant cost reduction and improved decision quality. Despite these promising results, the approach relied heavily on domain-specific datasets and high-performance computing infrastructure, limiting its generalizability across diverse software projects.

2.4.1.b Decision-Centric and Agile SDLC Models for AI Systems

As AI systems increasingly influence critical decision-making processes, researchers have emphasized the need for SDLC models that explicitly address decision quality, transparency, and governance. Decision-centric SDLC models extend traditional Agile methodologies by incorporating structured decision science principles.

[9]Gill (2025) proposed a decision-centric Agile SDLC framework specifically tailored for AI-based systems. The framework integrates decision architecture into Agile workflows to manage human and algorithmic bias, improve transparency, and strengthen governance. By treating decisions as first-class entities within the SDLC, the framework enhances accountability and trustworthiness in AI systems. However, the lack of empirical validation and quantitative performance metrics limits the framework's immediate applicability in industrial settings.

[10]Madupati (2024) examined the role of generative AI tools such as GPT-4 and OpenAI Codex in traditional SDLC models, including Waterfall and Agile. The study reported improved automation in code generation, debugging, testing, and CI/CD pipelines, leading to faster development cycles and enhanced productivity. Despite these advantages, the research raised critical concerns regarding ethical bias, over-reliance on AI-generated outputs, and limited compatibility with legacy systems, highlighting the need for balanced human-AI collaboration.

2.4.1.c Requirement Engineering Using AI and Machine Learning

Requirement engineering remains one of the most critical and error-prone phases of the SDLC, as ambiguities or inconsistencies introduced at this stage often propagate throughout the system lifecycle. Consequently, significant research effort has focused on applying AI and machine learning techniques to improve requirement clarity, consistency, and traceability.

[1]Ferrari et al. (2024) demonstrated that Large Language Models significantly enhance requirement interpretation, semantic understanding, and ambiguity detection

when compared to traditional NLP techniques. However, the study also revealed that LLMs are prone to hallucination when project-specific context is absent, resulting in unreliable outputs.

[4]Vineetha and Samuel (2022) employed a Multinomial Naïve Bayes classifier combined with NLP and Named Entity Recognition to extract actors and use cases from SRS documents. Although the approach achieved high accuracy and reduced manual effort, it lacked deep semantic reasoning and scalability across diverse domains.

[12]Zhang and Li (2024) further confirmed the superiority of LLMs over traditional NLP models for requirement classification tasks, while emphasizing challenges related to explainability, transparency, and cost efficiency.

2.4.1.d Design Automation and UML Generation

Design automation research aims to reduce manual modeling effort while ensuring alignment between requirements and system architecture. AI-driven design tools seek to generate UML diagrams and architectural artifacts directly from textual specifications.

[5]Alksasbeh et al. (2023) proposed a transformer-based UML generation pipeline that converts natural language requirements into intermediate semantic representations before generating UML models. While effective for functional requirements, the approach struggled to capture non-functional constraints and architectural quality attributes.

[13]Dias et al. (2023) developed an NLP-based system for generating UML use case diagrams from user stories. Although the system reduced manual effort, it faced limitations in handling complex relationships, spelling errors, and multilingual inputs.

[14]Cervantes et al. (2024) applied LLMs to software architecture design using Attribute-Driven Design (ADD). The approach improved documentation clarity and architectural decision traceability but suffered from context loss across iterative design cycles.

2.4.1.e Code Analysis, Testing, and Maintenance Automation

AI-driven automation has also been applied to later SDLC phases, including code review, testing, and maintenance.

[2]Raghi et al. (2024) explored LLM-assisted code review and reported improved developer productivity and faster review turnaround times. However, the study cautioned that AI-generated suggestions may introduce logical and security vulnerabilities if not carefully validated.

[15]Wang et al. (2023) provided a comprehensive survey of LLM-based software testing approaches and concluded that AI significantly improves test coverage and automation efficiency, while reinforcing the necessity of human-in-the-loop validation.

[16]Saravanan et al. (2025) demonstrated that generative AI improves debugging efficiency and code generation quality, but warned against excessive reliance on automation.

2.4.1.f Security-Aware SDLC and Security Debt

Security-aware SDLC research highlights that security considerations are often deprioritized in fast-paced development environments, leading to accumulated security debt.

[17]Boufaied (2025) investigated security debt through practitioner interviews and revealed that unresolved vulnerabilities frequently accumulate due to delivery pressure.

[18]Ghaleb (2025) emphasized that security tasks are commonly postponed in favor of feature delivery, allowing vulnerabilities to propagate across SDLC phases.

[19]Masood (2025) highlighted cautious adoption of AI-based security tools and stressed the importance of human validation to maintain accountability and trust.

2.4.1.g Ethics, Trust, and Human-in-the-Loop Systems

Ethical considerations, explainability, and trust are critical for the adoption of AI in software engineering.

[20]Russo (2023) analyzed organizational challenges in adopting generative AI and emphasized the importance of explainable and transparent AI systems.

[21]Jahić and Sami (2024) examined real-world applications of LLMs in software architecture and identified limitations in reasoning consistency.

[22]Sauvola et al. (2024) discussed long-term implications of generative AI, balancing productivity gains with ethical concerns.

[23]Belzner et al. (2023) demonstrated the effectiveness of LLMs across SDLC tasks while emphasizing the need for contextual grounding.

[24]Esposito et al. (2025) reviewed generative AI in software architecture and identified scalability and trustworthiness challenges.

TABLE 2.4: RESULTS COMPARISON OF RECENT AI-BASED SDLC STUDIES

Study	Reported Result	Improvement	Limitation
[1]Ferrari et al., 2024	Reduced documentation effort	~40%	Context dependency
[3]Arshad et al., 2025	Better defect prediction	+10–15% ROC-AUC	Dataset bias
[2]Raghi et al., 2024	Faster code review	Productivity gain	Validation needed
RAG studies, 2024	Reduced hallucination	Higher reliability	Complexity

While AI improves SDLC efficiency, results vary by task. RAG-based systems consistently show higher trustworthiness, making them suitable for enterprise SDLC environments.

TABLE 2.5: BENEFITS OF RAG-Based AI OVER DIRECT LLM USAGE

Aspect	Direct LLM	RAG-Based AI
Knowledge source	Pre-trained only	Project-specific
Hallucination	High	Low
Traceability	Not available	Available
Reliability	Medium	High
Suitability for SDLC	Limited	Strong

This table clearly justifies the selection of RAG for SDLC optimization by highlighting its advantages in accuracy, traceability, and reliability.

2.5 GAP IDENTIFIED FROM THE LITERATURE SURVEY

After synthesizing the reviewed studies, the following critical research gaps are identified:

1. Lack of end-to-end AI frameworks:
Most existing solutions address isolated SDLC phases rather than lifecycle-wide optimization.
2. Hallucination in LLM-only systems:
Direct LLM usage produces unreliable outputs without grounding.
3. Poor traceability:
Automated linkage between requirements, design, and code is rarely addressed.
4. Limited real-world evaluation:
Many studies rely on small or synthetic datasets.
5. Operational challenges:
Cost, latency, and privacy issues in cloud-based AI systems are underexplored.

2.5.1 Interpretation and Alignment with Proposed Work

These gaps directly motivate the proposed cloud-enabled, RAG-based AI framework, whose aim is to integrate multiple SDLC phases, ensure traceability, reduce hallucination, and support scalable deployment.

2.6 OVERALL OBSERVATIONS FROM THE LITERATURE

- AI significantly improves SDLC efficiency when properly integrated.
- Hybrid ML + LLM approaches outperform standalone models.
- RAG is essential for trustworthy AI in software engineering.
- Cloud infrastructure is critical for scalable SDLC optimization.

2.7 SUMMARY OF CHAPTER II

This chapter presented a comprehensive literature review of recent AI- and cloud-enabled approaches for optimizing the Software Development Life Cycle. Studies were analyzed thematically across SDLC phases and compared using structured tables. The review identified significant research gaps related to lifecycle integration, hallucination mitigation, and traceability. These gaps justify the need for a unified, cloud-enabled, RAG-based AI framework, which forms the foundation of the proposed system discussed in subsequent chapters.

CHAPTER III

SYSTEM REQUIREMENTS

This chapter describes the system requirements necessary for the design, development, and deployment of the proposed cloud-enabled, RAG-based AI framework for optimizing the Software Development Life Cycle (SDLC). The requirements are categorized into hardware requirements, software requirements, and a detailed explanation of the special software technologies used in the proposed system.

3.1 OVERVIEW OF SYSTEM REQUIREMENTS

The proposed framework is designed as a cloud-enabled AI-assisted decision-support system that integrates multiple SDLC activities such as requirement analysis, design support, testing assistance, and maintenance insights. To ensure efficient execution, scalability, and reliability, the system requires a combination of modern hardware resources and advanced software platforms. The requirements listed in this chapter are sufficient for both development and experimental deployment of the system.

3.2 HARDWARE REQUIREMENTS

The hardware requirements specify the minimum and recommended computing resources necessary to develop and run the proposed AI-based SDLC optimization framework efficiently.

3.2.1 Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	8 GB
Storage	256 GB SSD
System Type	64-bit architecture
Network	Stable internet connection
Input Devices	Keyboard, Mouse
Output Devices	Monitor

3.2.2 Recommended Hardware Requirements

Component	Specification
Processor	Intel Core i7 / AMD Ryzen 7 or higher
RAM	16 GB or above
Storage	512 GB SSD or above
GPU (Optional)	NVIDIA GPU with CUDA support (for model fine-tuning and embeddings)
Network	High-speed broadband internet

3.3 SOFTWARE REQUIREMENTS

The software requirements include the operating system, development tools, programming languages, cloud platforms, and AI frameworks required for implementing the proposed system.

3.3.1 Operating System

- Windows 10 / 11
- Ubuntu 20.04 or higher
- macOS (latest version)

3.3.2 Programming Languages

- Python – Core language for AI logic, RAG pipeline, and backend processing
- JavaScript – For frontend interaction and API integration (optional)
- HTML/CSS – For basic user interface design

3.3.3 Development Tools and Frameworks

- Python 3.9 or higher
- Flask / FastAPI – Backend API development
- VS Code / PyCharm – Development environment
- Git – Version control system

3.3.4 AI and NLP Libraries

- LangChain
- Hugging Face Transformers
- OpenAI / Azure OpenAI API
- SentenceTransformers

3.3.5 Database and Storage

- Vector Database: FAISS / ChromaDB / Pinecone
- Relational Database: PostgreSQL / MySQL (optional)
- Cloud Storage: AWS S3 / Azure Blob Storage

3.3.6 Cloud Platform

- Amazon Web Services (AWS) / Microsoft Azure / Google Cloud Platform
- Docker – Containerization
- Kubernetes (optional) – Orchestration

3.4 DETAILED EXPLANATION OF SPECIAL SOFTWARE USED

This section explains the key software technologies that play a critical role in implementing the proposed framework.

3.4.1 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation is the core technology used in the proposed system. RAG combines information retrieval and text generation to produce context-aware and reliable outputs. Instead of relying solely on a pre-trained language model, RAG retrieves relevant project-specific artifacts such as SRS documents, design files, and source code from a knowledge base and uses them as grounding context for generation.

This approach significantly reduces hallucination and improves traceability across SDLC phases.

3.4.2 Large Language Models (LLMs)

Large Language Models such as GPT-based architectures are used for understanding natural language requirements, generating design explanations, supporting test case generation, and providing maintenance insights. These models enable semantic reasoning and contextual understanding across software artifacts.

In the proposed system, LLMs act as decision-support agents rather than autonomous developers.

3.4.3 LangChain Framework

LangChain is used to orchestrate the interaction between retrieval mechanisms and language models. It enables chaining of prompts, integration with vector databases, and management of contextual memory, making it suitable for multi-stage SDLC workflows.

3.4.4 Vector Databases

Vector databases such as FAISS or ChromaDB store high-dimensional embeddings of requirements, design documents, and source code. These embeddings allow semantic similarity search, which is essential for retrieving relevant project context during RAG operations.

3.4.5 Cloud Infrastructure

Cloud platforms provide scalability, availability, and cost-effective resource management. The proposed system uses cloud services for hosting APIs, storing project artifacts, and accessing AI services. This ensures the framework can scale across multiple projects and teams.

3.4.6 Docker and Containerization

Docker is used to package the application and its dependencies into containers. This ensures consistent deployment across different environments and simplifies scalability and maintenance.

3.5 SUMMARY OF CHAPTER III

This chapter presented the complete system requirements for implementing the proposed cloud-enabled, RAG-based AI framework. It detailed the necessary hardware

and software components and provided an in-depth explanation of the special software technologies employed. These requirements ensure that the system is scalable, reliable, and capable of supporting intelligent assistance across multiple SDLC phases. The next chapter focuses on the system architecture and design of the proposed framework.

REFERENCES

- [1].Ferrari, A., Abualhaija, S., & Arora, C. (2024). Model Generation with LLMs: From Requirements to UML Sequence Diagrams. *2024 IEEE 32nd International Requirements Engineering Conference Workshops (REW)*, 291-300. <https://doi.org/10.1109/REW61692.2024.00044>.
- [2].Raghi, K., Sudha, K., M, S., & S, S. (2024). Software Development Automation Using Generative AI. *2024 International Conference on Emerging Research in Computational Science (ICERCS)*, 1-6. <https://doi.org/10.1109/ICERCS63125.2024.10894980>.
- [3].Arshad, N., Butt, T., & Iqbal, M. (2025). A Comprehensive Framework for Intelligent, Scalable, and Performance-Optimized Software Development. *IEEE Access*, 13, 74062-74077. <https://doi.org/10.1109/ACCESS.2025.3564139>.
- [4]. K, V., & Samuel, P. (2022). A Multinomial Naïve Bayes Classifier for identifying Actors and Use Cases from Software Requirement Specification documents. *2022 2nd International Conference on Intelligent Technologies (CONIT)*, 1-5. <https://doi.org/10.1109/CONIT55038.2022.9848290>.
- [5].Alksasbeh, M., Alqaralleh, B., Alramadin, T., & Alemerien, K. (2017). AN AUTOMATED USE CASE DIAGRAMS GENERATOR FROM NATURAL LANGUAGE REQUIREMENTS. .C., Li, A., Du, Y., Dong, H., & Yang, Y. (2024). Efficient and scalable reinforcement learning for large-scale network control. *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-024-00879-7>.
- [6].Butt, T., & Iqbal, M. (2025). A Comprehensive Framework for Intelligent, Scalable, and Performance-Optimized Software Development. *IEEE Access*, 13, 74062-74077. <https://doi.org/10.1109/ACCESS.2025.3564139>
- [7]. Iqbal, N., Sang, J., Chen, J., & Xia, X. (2021). Measuring Software Maintainability with Naïve Bayes Classifier. *Entropy*, 23. <https://doi.org/10.3390/e23020136>.
- [8].Faruqui, N., Thatoi, P., Choudhary, R., Roncevic, I., Alqahtani, H., Sarker, I., & Khanam, S. (2024). AI-Analyst: An AI-Assisted SDLC Analysis Framework for Business Cost Optimization. *IEEE Access*, 12, 195188-195203. <https://doi.org/10.1109/ACCESS.2024.3519423>.
- [9].Gill, A. (2025). Agile System Development Lifecycle for AI Systems: Decision Architecture. *ArXiv*, abs/2501.09434. <https://doi.org/10.48550/arXiv.2501.09434>
- [10].Madupati, B. (2025). AI's Impact on Traditional Software Development. *ArXiv*, abs/2502.18476. [https://doi.org/10.47363/JAICC/2024\(3\)E145](https://doi.org/10.47363/JAICC/2024(3)E145).

- [11].Zhang, Y., & Li, Y. (2024). The Classification of Software Requirements using Large Language Models: Emerging Results and Future Directions. *2024 3rd International Conference on Cloud Computing, Big Data Application and Software Engineering*
- [12].Dias, R., Vidanapathirana, C., Weerasinghe, R., Manupiya, A., Bandara, R., & Ranasinghe, Y. (2023). Automated use case diagram generator using NLP and ML. *ArXiv*, abs/2306.06962. <https://doi.org/10.48550/arXiv.2306.06962>
- [13].H. Cervantes et al., “An LLM-assisted approach to designing software architectures using ADD,” 2024. <https://doi.org/10.48550/arXiv.2506.22688>
(CBASE), 746-749. <https://doi.org/10.1109/CBASE64041.2024.10824444>.
- [14]. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2023). Software Testing With Large Language Models: Survey, Landscape, and Vision. *IEEE Transactions on Software Engineering*, 50, 911-936. <https://doi.org/10.1109/TSE.2024.3368208>.
- [15]. Saravanan, V., Kavitha, S., Ravi, S., Seetha, A., Rambabu, C., & Kanth, T. (2025). Generative AI in Software Engineering: Revolutionizing Code Generation and Debugging. *International Journal of Computational and Experimental Science and Engineering*. <https://doi.org/10.22399/ijcesen.1718>.
- [16].Boufaied, C., Ghaleb, T., & Masood, Z. (2025). Security Debt in Practice: Nuanced Insights from Practitioners. *arXiv preprint arXiv:2507.11362*. <https://arxiv.org/pdf/2507.11362>.
- [17]Ghaleb, T., & Masood, Z. (2025). Security Debt in Practice: Nuanced Insights from Practitioners. *arXiv preprint arXiv:2507.11362*. <https://arxiv.org/pdf/2507.11362>.
- [18]Masood, Z. (2025). Security Debt in Practice: Nuanced Insights from Practitioners. *arXiv preprint arXiv:2507.11362*. <https://arxiv.org/pdf/2507.11362>.
- [19].Russo, D. (2023). Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Transactions on Software Engineering and Methodology*, 33, 1 - 50. <https://doi.org/10.1145/3652154>.
- [20].Jahić, J., & Sami, A. (2024). State of Practice: LLMs in Software Engineering and Software Architecture. *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, 311-318. <https://doi.org/10.1109/ICSA-C63560.2024.00059>.
- [21].Sauvola, J., Tarkoma, S., Klemettinen, M., Riekk, J., & Doermann, D. (2024). Future of software development with generative AI. *Automated Software Engineering*, 31, 1-8. <https://doi.org/10.1007/s10515-024-00426-z>.
- [22]. Belzner, L., Gabor, T., & Wirsing, M. (2023). Large Language Model Assisted Software Engineering: Prospects, Challenges, and a Case Study. , 355-374. https://doi.org/10.1007/978-3-031-46002-9_23.

- [23].Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerný, T., Vaidhyanathan, K., Lenarduzzi, V., & Taibi, D. (2025). Generative AI for Software Architecture. Applications, Trends, Challenges, and Future Directions. *ArXiv*, abs/2503.13310. <https://doi.org/10.48550/arXiv.2503.13310>.
- [24]. Iqbal, N., Sang, J., Chen, J., & Xia, X. (2021). Measuring Software Maintainability with Naïve Bayes Classifier. *Entropy*, 23. <https://doi.org/10.3390/e23020136>.
- [25].Waqas Uzair, Sameen Naz. Six-Tier Architecture for AI-Generated Software Development: A Large Language Models Approach, 21 June 2023, PREPRINT (Version 1) available at Research Square <https://doi.org/10.21203/rs.3.rs-3086026/v1>.
- [26].Harzevili, N., & Alizadeh, S. (2018). Mixture of latent multinomial naive Bayes classifier. *Appl. Soft Comput.*, 69, 516-527. <https://doi.org/10.1016/j.asoc.2018.04.020>.
- [27].Bahı, A., Gharib, J., & Gahi, Y. (2024). Integrating Generative AI for Advancing Agile Software Development and Mitigating Project Management Challenges. *International Journal of Advanced Computer Science and Applications*. <https://doi.org/10.14569/ijacsa.2024.0150306>.
- [29].Alenezi, M., & Akour, M. (2025). AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Applied Sciences*. <https://doi.org/10.3390/app15031344>.
- [29].Mornie, M., Jali, N., Cheah, W., Mit, E., Saeed, S., Jali, S., & Greer, D. (2025). Visualisation of User Stories to UML use Case Diagram. *Journal of Advanced Research in Applied Sciences and Engineering Technology*. <https://doi.org/10.37934/araset.63.3.6880>.
- [30].Rädler, S., Berardinelli, L., Winter, K., Rahimi, A., & Rinderle-Ma, S. (2023). Bridging MDE and AI: a systematic review of domain-specific languages and model-driven practices in AI software systems engineering. *Softw. Syst. Model.*, 24, 445-469. <https://doi.org/10.1007/s10270-024-01211-y>.
- [31].Nair, R., & Thushara, M. (2025). NL2Code: A Hybrid NLP and Model-Driven Framework for Automated Code Generation from Natural Language and UML. 2025 *IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCEECS)*, 1-6. <https://doi.org/10.1109/SCEECS64059.2025.10940505>.
- [32].Ebert, C., Louridas, P., & Ebert, C. (2023). Generative AI for Software Practitioners. *IEEE Software*, 40, 30-38. <https://doi.org/10.1109/MS.2023.3265877>.
- [33].Kumar, R. (2025). Generative AI in Software Architecture: Transforming Design and Development Processes. *International Journal on Science and Technology*. <https://doi.org/10.71097/ijstat.v16.i1.3718>.
- [34].Kapoor, N. (2025). AI-Powered Data Insights in Product Lifecycle Management: Driving Innovation and Efficiency. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*. <https://doi.org/10.32628/cseit251112185>.

- [35].Fatima, A., Fernandes, A., Egan, D., & Luca, C. (2023). Software Requirements Prioritisation Using Machine Learning. , 893-900.<https://doi.org/10.5220/0011796900003393>.
- [36].Agarwal, P., Singhal, A., & Garg, A. (2017). SDLC Model Selection Tool and Risk Incorporation. *International Journal of Computer Applications*, 172, 6-10.
<https://doi.org/10.5120/ijca2017915143>.
- [37].Jüngling, S., Peraić, M., & Martin, A. (2020). Towards AI-based Solutions in the System Development Lifecycle.
- [38].Baldassarre, M., Gigante, D., Kalinowski, M., & Ragone, A. (2024). POLARIS: A framework to guide the development of Trustworthy AI systems. *2024 IEEE/ACM 3rd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 200-210.<https://doi.org/10.1145/3644815.3644947>.
- [39].Trinh, E., Funke, M., Lago, P., & Bogner, J. (2024). Sustainability Integration of Artificial Intelligence into the Software Development Life Cycle. *2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*, 199-206.
<https://doi.org/10.1109/ICSA-C63560.2024.00044>.
- [40].Dey, N., Ghosh, M., Kundu, S., & Chakrabarti, A. (2020). QDLC - The Quantum Development Life Cycle. *ArXiv*, abs/2010.08053.
- [41].Puranik, B., Sonawane, A., Jose, J., Chavan, S., & Patil, Y. (2024). Enhancement of Model Driven Software Development using AI. *2024 4th Asian Conference on Innovation in Technology (ASIANCON)*, 1-5.
<https://doi.org/10.1109/ASIANCON62057.2024.10837847>.
- [42].Ramackers, G., Griffioen, P., Schouten, M., & Chaudron, M. (2021). From Prose to Prototype: Synthesising Executable UML Models from Natural Language. *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*, 380-389.<https://doi.org/10.1109/models-c53483.2021.00061>.
- [43]. Enemosah, A. (2025). Enhancing DevOps Efficiency through AI-Driven Predictive Models for Continuous Integration and Deployment Pipelines. *International Journal of Research Publication and Reviews*.<https://doi.org/10.55248/gengpi.6.0125.0229>.
- [44]. Richardson, N., Kothapalli, S., Onteddu, A., Kundavaram, R., & Talla, R. (2023). AI- Driven Optimization Techniques for Evolving Software Architecture in Complex Systems. *ABC Journal of Advanced Research*.<https://doi.org/10.18034/abcjar.v12i2.783>.
- [45]. Ajiga, D., Okeleke, P., Folorunsho, S., & Ezeigweneme, C. (2024). Enhancing software development practices with AI insights in high-tech companies. *Computer Science & IT Research Journal*.<https://doi.org/10.51594/csitrj.v5i8.1450>.
- [46]. Wong, T., & Tsai, H. (2021). Multinomial naïve Bayesian classifier with generalized Dirichlet priors for high-dimensional imbalanced data. *Knowl. Based Syst.*, 228, 107288. <https://doi.org/10.1016/j.knosys.2021.107288>.

- [47]. Ruan, S., Li, H., Li, C., & Song, K. (2020). Class-Specific Deep Feature Weighting for Naïve Bayes Text Classifiers. *IEEE Access*, 8, 20151-20159. <https://doi.org/10.1109/ACCESS.2020.2968984>.
- [48]. Saklamaeva, V., & Pavlič, L. (2023). The Potential of AI-Driven Assistants in Scaled Agile Software Development. *Applied Sciences*. <https://doi.org/10.3390/app14010319>.
- [49]. Georgievski, I. (2023). Conceptualising Software Development Lifecycle for Engineering AI Planning Systems. *2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN)*, 88-89. <https://doi.org/10.1109/CAIN58948.2023.00019>.
- [50]. Elahi, M., Afolaranmi, S., Lastra, J., & García, J. (2023). A comprehensive literature review of the applications of AI techniques through the lifecycle of industrial equipment. *Discover Artificial Intelligence*, 3. <https://doi.org/10.1007/s44163-023-00089-x>.
- [51]. Barenkamp, M., Rebstadt, J., & Thomas, O. (2020). Applications of AI in classical software engineering. *AI Perspectives*, 2. <https://doi.org/10.1186/s42467-020-00005-4>.
- [52]. Marar, H. (2024). Advancements in software engineering using AI. *Computer Software and Media Applications*. <https://doi.org/10.24294/cзма.v6i1.3906>.
- [53]. Salehi, H., & Burgueño, R. (2018). Emerging artificial intelligence methods in structural engineering. *Engineering Structures*. <https://doi.org/10.1016/J.ENGSTRUCT.2018.05.084>.
- [54]. J, B., Ranjan, V., & Bhaduariya, N. (2023). Integration of AI in the Realm of Software Development. *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, 974-978. <https://doi.org/10.1109/ICAICCIT60255.2023.10465893>.
- [55]. Pangavhane, S., Raktate, G., Pariane, P., Shelar, K., Wakchaure, R., & Kale, J. (2024). AI-Augmented Software Development: Boosting Efficiency and Quality. *2024 International Conference on Decision Aid Sciences and Applications (DASA)*, 1-5. <https://doi.org/10.1109/DASA63652.2024.10836523>.
- [56]. Chen, S., Su, Y., Ku, Y., Lai, C., & Hsiao, K. (2022). Exploring the factors of students' intention to participate in AI software development. *Libr. Hi Tech*, 42, 392-408. <https://doi.org/10.1108/lht-12-2021-0480>.
- [57]. Pace, J., Tommasel, A., & Capilla, R. (2024). Helping Novice Architects to Make Quality Design Decisions Using an LLM-Based Assistant. , 324-332. https://doi.org/10.1007/978-3-031-70797-1_21.
- [58]. Zamfirescu-Pereira, J., Jun, E., Terry, M., Yang, Q., & Hartmann, B. (2025). Beyond Code Generation: LLM-supported Exploration of the Program Design Space. *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. <https://doi.org/10.1145/3706598.3714154>.

- [59]. Zhou, L., Lv, Z., Zhang, X., Hu, J., Pan, L., Pang, X., & Shi, J. (2025). Research on AI-assisted design method of substation based on LLM+RAG. *IET Conference Proceedings*. <https://doi.org/10.1049/icp.2024.4307>.
- [60]. Maatuk, A., & Abdelnabi, E. (2021). Generating UML Use Case and Activity Diagrams Using NLP Techniques and Heuristics Rules. *International Conference on Data Science, E-learning and Information Systems 2021*. <https://doi.org/10.1145/3460620.3460768>.
- [61]. Santos, J., Moshtari, S., & Mirakhorli, M. (2020). An Automated Approach to Recover the Use-case View of an Architecture. *2020 IEEE International Conference on Software Architecture Companion (ICSA-C)*, 63-66. <https://doi.org/10.1109/ICSA-C50368.2020.00020>.
- [62]. Narawita, C., & Vidanage, K. (2018). UML generator – use case and class diagram generation from text requirements. *International Journal on Advances in Ict for Emerging Regions (icter)*, 10, 1. <https://doi.org/10.4038/ICTER.V10I1.7182>.
- [63]. Iyengar, P., Otte, F., & Pulvermueller, E. (2022). AI-guided Model-Driven Embedded Software Engineering. , 395-404. <https://doi.org/10.5220/0011006200003119>.
- [64]. Srivastava, S. (2024). AI-Enhanced Automation for DevOps: Employing a Model-Driven Strategy to Facilitate Continuous Advancement in Cyber-Physical Systems. *International Journal of Science and Research (IJSR)*. <https://doi.org/10.21275/sr24226073411>.
- [65]. (2021). Model Development. *Artificial Intelligence for Asset Management and Investment*. <https://doi.org/10.1002/9780470258019.ch5>.
- [66]. Deeptimahanti, D., & Babar, M. (2009). An Automated Tool for Generating UML Models from Natural Language Requirements. *2009 IEEE/ACM International Conference on Automated Software Engineering*, 680-682. <https://doi.org/10.1109/ASE.2009.48>.
- [67]. Abdelnabi, E., Maatuk, A., & Abdelaziz, T. (2021). An Algorithmic Approach for Generating Behavioral UML Models Using Natural Language Processing. *The 7th International Conference on Engineering & MIS 2021*. <https://doi.org/10.1145/3492547.3492612>.
- [68]. Staron, M., Abrahão, S., & Abrahão, S. (2025). Exploring Generative AI in Automated Software Engineering. *IEEE Software*, 42, 142-145. <https://doi.org/10.1109/MS.2025.3533754>.
- [69]. Saravanan, V., Kavitha, S., Ravi, S., Seetha, A., Rambabu, C., & Kanth, T. (2025). Generative AI in Software Engineering: Revolutionizing Code Generation and Debugging. *International Journal of Computational and Experimental Science and Engineering*. <https://doi.org/10.22399/ijcesen.1718>.
- [70]. Ooi, K., Tan, G., Al-Emran, M., Al-Sharafi, M., Căpățînă, A., Chakraborty, A., Dwivedi, Y., Huang, T., Kar, A., Lee, V., Loh, X., Micu, A., Mikalef, P., Mogaji, E., Pandey, N., Raman, R., Rana, N., Sarker, P., Sharma, A., Teng, C., Wamba, S., & Wong,

- L. (2023). The Potential of Generative Artificial Intelligence Across Disciplines: Perspectives and Future Directions. *Journal of Computer Information Systems*, 65, 76 - 107. <https://doi.org/10.1080/08874417.2023.2261010>.
- [71]. Sarma, W., Tiwari, S., & Dey, S. (2023). Architecting Next-Generation Software Systems with Generative AI and Large Language Models: Challenges, Opportunities, and Best Practices. *International Journal of Innovative Research in Computer and Communication Engineering*. <https://doi.org/10.15680/ijircce.2023.1105367>.
- [72]. Al-Kfairy, M., Mustafa, D., Kshetri, N., Insiew, M., & Alfandi, O. (2024). Ethical Challenges and Solutions of Generative AI: An Interdisciplinary Perspective. *Informatics*, 11, 58. <https://doi.org/10.3390/informatics11030058>.
- [73]. Hummer, W., Muthusamy, V., Rausch, T., Dube, P., Maghraoui, K., Murthi, A., & Oum, P. (2019). ModelOps: Cloud-Based Lifecycle Management for Reliable and Trusted AI. *2019 IEEE International Conference on Cloud Engineering (IC2E)*, 113-120. <https://doi.org/10.1109/IC2E.2019.00025>.
- [74]. Ullrich, L., Buchholz, M., Dietmayer, K., & Graichen, K. (2025). Expanding the Classical V-Model for the Development of Complex Systems Incorporating AI. *ArXiv*, abs/2502.13184. <https://doi.org/10.1109/TIV.2024.3434515>.
- [75]. Golani, R. (2025). Leveraging AI to Scale Product Development: Technical Approaches and Implementation Strategies. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*. <https://doi.org/10.32628/cseit25112742>.
- [76]. Jahan, M., Hassan, M., Golpayegani, R., Ranjbaran, G., Roy, C., Roy, B., & Schneider, K. (2024). Automated Derivation of UML Sequence Diagrams from User Stories: Unleashing the Power of Generative AI vs. a Rule-Based Approach. *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*. <https://doi.org/10.1145/3640310.3674081>.
- [77]. De Bari, D., Garaccione, G., Coppola, R., Torchiano, M., & Ardito, L. (2024). Evaluating Large Language Models in Exercises of UML Class Diagram Modeling. *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*. <https://doi.org/10.1145/3674805.3690741>.
- [78]. Rocha, M., Simão, A., & Sousa, T. (2021). Model-based test case generation from UML sequence diagrams using extended finite state machines. *Software Quality Journal*, 29, 597 - 627. <https://doi.org/10.1007/s11219-020-09531-0>.
- [79]. Jahan, M., Abad, Z., & Far, B. (2021). Generating Sequence Diagram from Natural Language Requirements. *2021 IEEE 29th International Requirements Engineering Conference Workshops (REW)*, 39-48. <https://doi.org/10.1109/REW53955.2021.00012>.
- [80]. Li, B., Qi, P., Liu, B., Di, S., Liu, J., Pei, J., Yi, J., & Zhou, B. (2021). Trustworthy AI: From Principles to Practices. *ACM Computing Surveys*, 55, 1 - 46. <https://doi.org/10.1145/3555803>.

- [81]. Kemmerzell, N., Schreiner, A., Khalid, H., Schalk, M., & Bordoli, L. (2025). Towards a Better Understanding of Evaluating Trustworthiness in AI Systems. *ACM Computing Surveys*. <https://doi.org/10.1145/3721976>.
- [82]. Pelosi, A., Felicioli, C., Canciani, A., & Severino, F. (2023). A Hybrid-DLT Based Trustworthy AI Framework. *2023 IEEE International Conference on Enabling Technologies: Infrastructure for Collaborative Enterprises (WETICE)*, 1-6. <https://doi.org/10.1109/WETICE57085.2023.10477792>.
- [83]. Stettinger, G., Weissensteiner, P., & Khastgir, S. (2024). Trustworthiness Assurance Assessment for High-Risk AI-Based Systems. *IEEE Access*, 12, 22718-22745. <https://doi.org/10.1109/ACCESS.2024.3364387>.
- [84]. Adeagbo, M., Akinsola, J., Awoseyi, A., & Kasali, F. (2021). Project Implementation Decision Using Software Development Life Cycle Models: A Comparative Approach. *Journal of Computer Science and Its Application*. <https://doi.org/10.4314/jcsia.v28i1.10>.
- [85]. Olusanya, O., Jimoh, R., Misra, S., & Awotunde, J. (2024). A neuro-fuzzy security risk assessment system for software development life cycle. *Heliyon*, 10. <https://doi.org/10.1016/j.heliyon.2024.e33495>.
- [86]. Khan, P., & Beg, M. (2013). Extended Decision Support Matrix for Selection of SDLC-Models on Traditional and Agile Software Development Projects. *2013 Third International Conference on Advanced Computing and Communication Technologies (ACCT)*, 8-15. <https://doi.org/10.1109/ACCT.2013.12>.
- [87]. Binsawad, M., & Khan, B. (2024). FEPP: Advancing Software Risk Prediction in Requirements Engineering Through Innovative Rule Extraction and Multi-Class Integration. *IEEE Access*, 12, 59851-59860. <https://doi.org/10.1109/ACCESS.2024.3392283>.
- [88]. Kruke, M., Martini, A., Cruzes, D., & Iovan, M. (2024). Defining Security Debt: A Case Study Based on Practice. , 43-59. https://doi.org/10.1007/978-3-031-78386-9_4.
- [89]. Martinez, J., Quintano, N., Ruiz, A., Santamaría, I., De Soria, I., & Arias, J. (2021). Security Debt: Characteristics, Product Life-Cycle Integration and Items. *2021 IEEE/ACM International Conference on Technical Debt (TechDebt)*, 1-5. <https://doi.org/10.1109/TechDebt52882.2021.00009>.
- [90]. Marsh, P. (1982). The Choice Between Equity and Debt: An Empirical Study. *Journal of Finance*, 37, 121-144. <https://doi.org/10.1111/J.1540-6261.1982.TB01099.X>.
- [91]. Koilo, V., Ryabushka, L., Kubakh, T., & Halik, J. (2020). Assessment of Government Debt Security of Emerging Markets: Theory and Practice. *ERN: Other Emerging Markets Economics: Macroeconomic Issues & Challenges (Topic)*. [https://doi.org/10.21511/imfi.17\(1\).2020.04](https://doi.org/10.21511/imfi.17(1).2020.04).
- [92]. Li, X., Ye, P., Li, J., Liu, Z., Cao, L., Wang, F., & Wang, F. (2022). From Features Engineering to Scenarios Engineering for Trustworthy AI: I&I, C&C, and V&V. *IEEE Intelligent Systems*, 37, 18-26. <https://doi.org/10.1109/MIS.2022.3197950>.

- [93]. Randhi, K., & Bandarapu, S. (2024). Efficient resource allocation for generative AI workloads in cloud-native infrastructures: A multi-tiered approach. *International Journal of Science and Research Archive*. <https://doi.org/10.30574/ijrsra.2024.13.2.2208>.
- [94]. Jon, W. (2025). Prompting Creativity: Tiered Approach to Copyright Protection for AI-Generated Content in the Digital Age. *Media and Communication*. <https://doi.org/10.17645/mac.9420>.
- [95]. Perini, A., Susi, A., & Avesani, P. (2013). A Machine Learning Approach to Software Requirements Prioritization. *IEEE Transactions on Software Engineering*, 39, 445-461. <https://doi.org/10.1109/TSE.2012.52>.
- [96]. Talele, P., & Phalnikar, R. (2020). Software Requirements Classification and Prioritisation Using Machine Learning. , 257-267. https://doi.org/10.1007/978-981-15-7106-0_26.
- [97]. Talele, P., & Phalnikar, R. (2021). Classification and Prioritisation of Software Requirements using Machine Learning – A Systematic Review. *2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, 912-918. <https://doi.org/10.1109/Confluence51648.2021.9377190>.
- [98]. Berhanu, F., & Alemneh, E. (2023). Classification and Prioritization of Requirements Smells Using Machine Learning Techniques. *2023 International Conference on Information and Communication Technology for Development for Africa (ICT4DA)*, 49-54. <https://doi.org/10.1109/ICT4DA59526.2023.10302263>.
- [99]. Mallampati, B. (2025). The role of generative AI in software development: Will it replace developers?. *World Journal of Advanced Research and Reviews*. <https://doi.org/10.30574/wjarr.2025.26.1.1387>.
- [100]. A. (2024). Generative Ai in Software Development : an Overview and Evaluation of Modern Coding Tools. *International Journal For Multidisciplinary Research*. <https://doi.org/10.36948/ijfmr.2024.v06i03.23271>.
- [101]. Krishna, K., Murthy, P., & Sarangi, S. (2024). Exploring the synergy between generative AI and software engineering: Automating code optimization and bug fixing. *World Journal of Advanced Engineering Technology and Sciences*. <https://doi.org/10.30574/wjaets.2024.13.1.0464>.
- [102]. Soureya, Y., Ngoumou, A., Ngossaha, J., Tsakou, S., & Ndjodo, M. (2025). Adaptive software development: a comprehensive framework integrating artificial intelligence for sustainable evolution. *Int. Arab J. Inf. Technol.*, 22, 248-262. <https://doi.org/10.34028/iajit/22/2/4>.
- [103]. Goralski, M., & Tan, T. (2020). Artificial intelligence and sustainable development. *The International Journal of Management Education*. <https://doi.org/10.1016/j.ijme.2019.100330>.
- [104]. Kulkov, I., Kulkova, J., Rohrbeck, R., Menvielle, L., Kaartemo, V., & Makkonen, H. (2023). Artificial intelligence - driven sustainable development: Examining

organizational, technical, and processing approaches to achieving global goals. *Sustainable Development*. <https://doi.org/10.1002/sd.2773>.

[105]. Bolón-Canedo, V., Morán-Fernández, L., Cancela, B., & Alonso-Betanzos, A. (2024). A review of green artificial intelligence: Towards a more sustainable future. *Neurocomputing*, 599, 128096. <https://doi.org/10.1016/j.neucom.2024.128096>.

[106]. Weder, B., Barzen, J., Leymann, F., Salm, M., & Vietz, D. (2020). The Quantum software lifecycle. *Proceedings of the 1st ACM SIGSOFT International Workshop on Architectures and Paradigms for Engineering Quantum Software*. <https://doi.org/10.1145/3412451.3428497>.

[107]. Pérez-Castillo, R., Serrano, M., Cruz-Lemus, J., & Piattini, M. (2024). Guidelines to use the ICSM for developing quantum-classical systems. *Quantum Inf. Comput.*, 24, 71-88. <https://doi.org/10.26421/QIC24.1-2-4>.

[108]. Coccia, M., Roshani, S., & Mosleh, M. (2024). Evolution of Quantum Computing: Theoretical and Innovation Management Implications for Emerging Quantum Industry. *IEEE Transactions on Engineering Management*, 71, 2270-2280. <https://doi.org/10.1109/TEM.2022.3175633>.

[109]. Nagarajan, S., Malarvel, M., & J, T. (2024). A Framework for Quantum based Software Development Process. *2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS)*, 1-6. <https://doi.org/10.1109/ADICS58448.2024.10533595>.

[110]. Saqib, M., Mustaqeem, M., Jawed, M., Abdulaziz, A., Khan, A., & Khan, J. (2024). Deep-transfer learning inspired natural language processing system for software requirements classification. *Knowl. Inf. Syst.*, 67, 839-861. <https://doi.org/10.1007/s10115-024-02248-7>.

[111]. Van Viet, N., & Vinh, N. (2024). Large language models in software engineering. *Journal of Education For Sustainable Innovation*. <https://doi.org/10.56916/jesi.v2i2.968>.